
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 8

Caracteres e Strings em C

Exercícios (8.5–8.42)

Resolvidos passo a passo, com código completo

Contents

1	Exercício 8.5 – Teste de Caracteres	2
2	Exercício 8.6 – Maiúsculas e Minúsculas	2
3	Exercício 8.7 – Conversão de Strings em Inteiros	3
4	Exercício 8.8 – Conversão para double	4
5	Exercício 8.9 – Comparação de Strings	5
6	Exercício 8.10 – strcmp	5
7	Exercício 8.11 – Sentenças Aleatórias	7
8	Exercício 8.12 – Limericks	8
9	Exercício 8.13 – Falso Latim (Pig Latin)	9
10	Exercício 8.14 – Tokenização de Telefone	10
11	Exercício 8.15 – Inverter Ordem das Palavras	11
12	Exercício 8.16 – Procurar Substring	11
13	Exercício 8.17 – Total de Ocorrências de Substring	13
14	Exercício 8.18 – Total de Ocorrências de Caractere	13
15	Exercício 8.19 – Frequência das Letras do Alfabeto	14
16	Exercício 8.20 – Total de Palavras	16
17	Exercício 8.21 – Ordenação Alfabética de Strings	16
18	Exercício 8.22 – ASCII (V/F)	18
19	Exercício 8.23 – Strings que começam com 'b'	18
20	Exercício 8.24 – Strings terminando em 'ed'	19
21	Exercício 8.25 – Tabela ASCII Completa	20
22	Exercícios 8.26 a 8.33 – Reimplementar Funções da Biblioteca	20
23	Exercício 8.34 – Análise de Texto	22
24	Exercício 8.35 – Justificação de Texto	24
25	Exercício 8.36 – Datas em Vários Formatos	24
26	Exercício 8.37 – Proteção de Cheque	26

27 Exercício 8.38 – Valor de Cheque por Extenso	27
28 Exercício 8.39 – Código Morse	28
29 Exercício 8.40 – Conversor Métrico	30
30 Exercício 8.41 – Cartas de Cobrança	30
31 Exercício 8.42 – Gerador de Palavras Cruzadas	31

1. Exercício 8.5 – Teste de Caracteres

Resposta detalhada

```
1  /* Ex8.5: teste_caracteres.c
2     Aplica todas as funcoes de <ctype.h> a um caractere
3     */
4  #include <stdio.h>
5  #include <ctype.h>
6
7  int main( void )
8  {
9
10     int c;
11
12     printf( "Digite um caractere: " );
13     c = getchar();
14
15     printf( "\nResultados para o caractere '%c' (ASCII
16           %d):\n", c, c );
17     printf( "isdigit   : %d\n", isdigit( c ) );
18     printf( "isalpha   : %d\n", isalpha( c ) );
19     printf( "isalnum   : %d\n", isalnum( c ) );
20     printf( "isxdigit  : %d\n", isxdigit( c ) );
21     printf( "islower   : %d\n", islower( c ) );
22     printf( "isupper   : %d\n", isupper( c ) );
23     printf( "isspace   : %d\n", isspace( c ) );
24     printf( "iscntrl   : %d\n", iscntrl( c ) );
25     printf( "isprint   : %d\n", isprint( c ) );
26     printf( "ispunct   : %d\n", ispunct( c ) );
27     printf( "tolower   : '%c'\n", tolower( c ) );
28     printf( "toupper   : '%c'\n", toupper( c ) );
29
30     return 0;
31 }
```

2. Exercício 8.6 – Maiúsculas e Minúsculas

Resposta detalhada

```
1  /* Ex8.6: maiusculas_minusculas.c */
2  #include <stdio.h>
3  #include <ctype.h>
4  #include <string.h>
5
6  int main( void )
7  {
8     char s[ 100 ];
9     int i;
```

```
10
11     printf( "Digite uma linha de texto: " );
12     fgets( s, 100, stdin );
13
14     printf( "Maiusculas: " );
15     for ( i = 0; s[ i ] != '\0'; ++i ) {
16         putchar( toupper( s[ i ] ) );
17     }
18
19     printf( "Minusculas: " );
20     for ( i = 0; s[ i ] != '\0'; ++i ) {
21         putchar( tolower( s[ i ] ) );
22     }
23
24     return 0;
25 }
```

3. Exercício 8.7 – Conversão de Strings em Inteiros

Resposta detalhada

```
1  /* Ex8.7: soma_inteiros_string.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main( void )
6  {
7      char s1[ 20 ], s2[ 20 ], s3[ 20 ], s4[ 20 ];
8      int v1, v2, v3, v4, total;
9
10     printf( "Digite 4 strings com inteiros (uma por
11             linha):\n" );
12     scanf( "%s%s%s%s", s1, s2, s3, s4 );
13
14     v1 = atoi( s1 );
15     v2 = atoi( s2 );
16     v3 = atoi( s3 );
17     v4 = atoi( s4 );
18
19     total = v1 + v2 + v3 + v4;
20     printf( "Soma: %d + %d + %d + %d = %d\n", v1, v2,
21             v3, v4, total );
22
23     return 0;
24 }
```

4. Exercício 8.8 – Conversão para double

Resposta detalhada

```
1  /* Ex8.8: soma_double_string.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main( void )
6  {
7      char s1[ 20 ], s2[ 20 ], s3[ 20 ], s4[ 20 ];
8      double v1, v2, v3, v4, total;
9
10     printf( "Digite 4 strings com valores double:\n" )
11         ;
12     scanf( "%s%s%s%s", s1, s2, s3, s4 );
13
14     v1 = atof( s1 );
15     v2 = atof( s2 );
16     v3 = atof( s3 );
17     v4 = atof( s4 );
18
19     total = v1 + v2 + v3 + v4;
20     printf( "Soma: %.2f + %.2f + %.2f + %.2f = %.2f\n"
21         ,
22         v1, v2, v3, v4, total );
23
24     return 0;
25 }
```

5. Exercício 8.9 – Comparação de Strings

Resposta detalhada

```
1  /* Ex8.9: compara_strings.c */
2  #include <stdio.h>
3  #include <string.h>
4
5  int main( void )
6  {
7      char s1[ 100 ], s2[ 100 ];
8      int resultado;
9
10     printf( "Digite a primeira string: " );
11     fgets( s1, 100, stdin );
12     s1[ strcspn( s1, "\n" ) ] = '\0'; /* remove \n do
        fgets */
13
14     printf( "Digite a segunda string: " );
15     fgets( s2, 100, stdin );
16     s2[ strcspn( s2, "\n" ) ] = '\0';
17
18     resultado = strcmp( s1, s2 );
19
20     if ( resultado < 0 ) {
21         printf( "\"%s\" e menor que \"%s\"\n", s1, s2
22             );
23     }
24     else if ( resultado == 0 ) {
25         printf( "As strings sao iguais.\n" );
26     }
27     else {
28         printf( "\"%s\" e maior que \"%s\"\n", s1, s2
29             );
30     }
31
32     return 0;
33 }
```

6. Exercício 8.10 – strncmp

Resposta detalhada

```
1  /* Ex8.10: compara_n_chars.c */
2  #include <stdio.h>
3  #include <string.h>
4
5  int main( void )
```

```
6  {
7      char s1[ 100 ], s2[ 100 ];
8      int n, resultado;
9
10     printf( "Primeira string: " );
11     fgets( s1, 100, stdin );
12     s1[ strcspn( s1, "\n" ) ] = '\0';
13
14     printf( "Segunda string: " );
15     fgets( s2, 100, stdin );
16     s2[ strcspn( s2, "\n" ) ] = '\0';
17
18     printf( "Numero de caracteres a comparar: " );
19     scanf( "%d", &n );
20
21     resultado = strncmp( s1, s2, n );
22
23     if ( resultado < 0 )
24         printf( "Primeira %d chars de s1 sao MENORES.\n", n );
25     else if ( resultado == 0 )
26         printf( "Primeira %d chars sao IGUAIS.\n", n );
27     else
28         printf( "Primeira %d chars de s1 sao MAIORES.\n", n );
29
30     return 0;
31 }
```


7. Exercício 8.11 – Sentenças Aleatórias

Resposta detalhada

```
1  /* Ex8.11: sentencas_aleatorias.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <ctype.h>
6  #include <time.h>
7
8  int main( void )
9  {
10     const char *artigo[]      = { "o", "a", "um", "uma",
11                                   "algum", "alguma" };
12     const char *substantivo[] = { "menino", "menina",
13                                   "cao", "cidade", "carro" };
14     const char *verbo[]       = { "dirigiu", "saltou",
15                                   "correu", "caminhou", "pulou" };
16     const char *preposicao[]   = { "para", "de", "sobre",
17                                   "sob", "em" };
18
19     char sentenca[ 200 ];
20     int i, frase;
21
22     srand( time( NULL ) );
23
24     for ( frase = 1; frase <= 20; ++frase ) {
25         sentenca[ 0 ] = '\0'; /* string vazia */
26
27         strcat( sentenca, artigo[ rand() % 6 ] );
28         strcat( sentenca, " " );
29         strcat( sentenca, substantivo[ rand() % 5 ] );
30         strcat( sentenca, " " );
31         strcat( sentenca, verbo[ rand() % 5 ] );
32         strcat( sentenca, " " );
33         strcat( sentenca, preposicao[ rand() % 5 ] );
34         strcat( sentenca, " " );
35         strcat( sentenca, artigo[ rand() % 6 ] );
36         strcat( sentenca, " " );
37         strcat( sentenca, substantivo[ rand() % 5 ] );
38         strcat( sentenca, "." );
39
40         /* Capitaliza primeira letra */
41         sentenca[ 0 ] = toupper( sentenca[ 0 ] );
42
43         printf( "%s\n", sentenca );
44     }
45
46     return 0;
47 }
```

38 }

Exemplo de saída:**Saída do programa**

```
Um menino correu sobre a cidade.  
Alguma menina dirigiu para o cao.  
O carro caminhou em uma menina.  
...
```

Conceito-chave: usamos um **array de ponteiros para char** (arrays de strings literais), técnica do Cap. 7 que dispensa pré-alocação de matrizes 2D.

8. Exercício 8.12 – Limericks

Resposta detalhada

Estratégia: Generalizar o Ex. 8.11 com 5 conjuntos de palavras agrupados por padrão de rima (A-A-B-B-A). Cada linha do limerick tem estrutura própria.

```
1  /* Esqueleto de limerick generator */  
2  const char *linhasA[] = { /* terminam em rima A */  
3      "Conheci uma menina chamada Sue",  
4      "Vivia num lugar que era cru",  
5      /* ... mais opcoes ... */  
6  };  
7  const char *linhasB[] = { /* rima B */  
8      "Tinha um cao que rosnava",  
9      "Andava sem ter mapa",  
10     /* ... */  
11 };  
12  
13 /* Imprime: linhaA1, linhaA2, linhaB1, linhaB2,  
    linhaA3 */
```

Desafio: fazer as rimas funcionarem requer cuidado linguístico. Esta é uma das tarefas mais difíceis em geração de texto – até IA moderna luta para fazer limericks consistentes em português.

9. Exercício 8.13 – Falso Latim (Pig Latin)

Resposta detalhada

Algoritmo: Para cada palavra, passar a primeira letra para o final e adicionar “ei”. Exemplo: “salto” → “altosei”.

```
1  /* Ex8.13: pig_latin.c */
2  #include <stdio.h>
3  #include <string.h>
4
5  void printLatinWord( char *palavra );
6
7  int main( void )
8  {
9      char frase[ 200 ];
10     char *token;
11
12     printf( "Digite uma frase: " );
13     fgets( frase, 200, stdin );
14     frase[ strcspn( frase, "\n" ) ] = '\0';
15
16     token = strtok( frase, " " );
17     while ( token != NULL ) {
18         printLatinWord( token );
19         printf( " " );
20         token = strtok( NULL, " " );
21     }
22     printf( "\n" );
23
24     return 0;
25 }
26
27 void printLatinWord( char *palavra )
28 {
29     char primeira = palavra[ 0 ];
30     int tam      = strlen( palavra );
31     int i;
32
33     /* Imprime a partir do segundo caractere */
34     for ( i = 1; i < tam; ++i ) {
35         putchar( palavra[ i ] );
36     }
37
38     /* Adiciona a primeira letra + "ei" */
39     putchar( primeira );
40     printf( "ei" );
41 }
```

Exemplo: "o computador salta" → "oei omputadorcei altasei".

10. Exercício 8.14 – Tokenização de Telefone

Resposta detalhada

```
1  /* Ex8.14: tokens_telefone.c
2     Le "(55) 5555-5555" e extrai DDD e numero */
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6
7  int main( void )
8  {
9     char telefone[ 30 ];
10    char numero[ 9 ] = { 0 };
11    char *token;
12    int ddd;
13    long numTel;
14
15    printf( "Digite o telefone como (DD) XXXX-XXXX: "
16           );
17    fgets( telefone, 30, stdin );
18
19    /* Primeiro token: depois de '(' */
20    token = strtok( telefone, "() -\n" ); /* DDD */
21    ddd = atoi( token );
22
23    token = strtok( NULL, "() -\n" ); /* primeira
24    parte */
25    strcat( numero, token );
26
27    token = strtok( NULL, "() -\n" ); /* segunda
28    parte */
29    strcat( numero, token );
30
31    numTel = atol( numero );
32
33    printf( "DDD: %d\n", ddd );
34    printf( "Numero: %ld\n", numTel );
35
36    return 0;
37 }
```

11. Exercício 8.15 – Inverter Ordem das Palavras

Resposta detalhada

Estratégia: extrair tokens, armazenar em array, imprimir em ordem inversa.

```
1  /* Ex8.15: inverter_palavras.c */
2  #include <stdio.h>
3  #include <string.h>
4  #define MAX_PALAVRAS 50
5
6  int main( void )
7  {
8      char texto[ 500 ];
9      char *palavras[ MAX_PALAVRAS ];
10     char *token;
11     int n = 0, i;
12
13     printf( "Digite uma linha de texto: " );
14     fgets( texto, 500, stdin );
15
16     /* Extrai tokens */
17     token = strtok( texto, " \n" );
18     while ( token != NULL && n < MAX_PALAVRAS ) {
19         palavras[ n++ ] = token;
20         token = strtok( NULL, " \n" );
21     }
22
23     /* Imprime de tras para a frente */
24     printf( "Invertido: " );
25     for ( i = n - 1; i >= 0; --i ) {
26         printf( "%s ", palavras[ i ] );
27     }
28     printf( "\n" );
29
30     return 0;
31 }
```

12. Exercício 8.16 – Procurar Substring

Resposta detalhada

```
1  /* Ex8.16: procura_substring.c */
2  #include <stdio.h>
3  #include <string.h>
4
5  int main( void )
6  {
7      char linha[ 200 ], procura[ 50 ];
```

```
8      char *searchPtr;
9
10     printf( "Digite a linha de texto: " );
11     fgets( linha, 200, stdin );
12
13     printf( "Digite a string de pesquisa: " );
14     fgets( procura, 50, stdin );
15     procura[ strcspn( procura, "\n" ) ] = '\0';
16
17     searchPtr = strstr( linha, procura );
18
19     if ( searchPtr != NULL ) {
20         printf( "Primeira ocorrencia em diante:\n%s\n"
21             , searchPtr );
22
23         /* Procura segunda ocorrencia */
24         searchPtr = strstr( searchPtr + 1, procura );
25         if ( searchPtr != NULL ) {
26             printf( "\nSegunda ocorrencia em diante:\n
27                 %s\n", searchPtr );
28         }
29         else {
30             printf( "\nNao ha segunda ocorrencia.\n" )
31                 ;
32         }
33     }
34     else {
35         printf( "String nao encontrada.\n" );
36     }
37
38     return 0;
39 }
```

13. Exercício 8.17 – Total de Ocorrências de Substring

Resposta detalhada

```
1  /* Ex8.17: conta_substring.c */
2  #include <stdio.h>
3  #include <string.h>
4
5  int main( void )
6  {
7      char texto[ 500 ];
8      char procura[ 50 ];
9      int total = 0;
10     char *p;
11
12     printf( "Digite o texto (em uma linha): " );
13     fgets( texto, 500, stdin );
14
15     printf( "Digite a substring a procurar: " );
16     fgets( procura, 50, stdin );
17     procura[ strcspn( procura, "\n" ) ] = '\0';
18
19     p = texto;
20     while ( ( p = strstr( p, procura ) ) != NULL ) {
21         ++total;
22         ++p; /* avanca para procurar proxima
23              ocorrencia */
24     }
25
26     printf( "Ocorrencias de '%s': %d\n", procura,
27            total );
28     return 0;
29 }
```

14. Exercício 8.18 – Total de Ocorrências de Caractere

Resposta detalhada

```
1  /* Ex8.18: conta_caractere.c */
2  #include <stdio.h>
3  #include <string.h>
4
5  int main( void )
6  {
7      char texto[ 500 ];
8      char ch;
9      int total = 0;
10     char *p;
```

```

11
12     printf( "Digite o texto: " );
13     fgets( texto, 500, stdin );
14
15     printf( "Digite o caractere a procurar: " );
16     scanf( " %c", &ch );
17
18     p = texto;
19     while ( ( p = strchr( p, ch ) ) != NULL ) {
20         ++total;
21         ++p;
22     }
23
24     printf( "'%c' ocorre %d vezes.\n", ch, total );
25     return 0;
26 }

```

15. Exercício 8.19 – Frequência das Letras do Alfabeto

Resposta detalhada

```

1  /* Ex8.19: frequencia_letras.c */
2  #include <stdio.h>
3  #include <ctype.h>
4  #include <string.h>
5
6  int main( void )
7  {
8      char texto[ 500 ];
9      int frequencia[ 26 ] = { 0 };
10     int i;
11     char letra;
12
13     printf( "Digite o texto: " );
14     fgets( texto, 500, stdin );
15
16     /* Conta cada letra (case-insensitive) */
17     for ( i = 0; texto[ i ] != '\0'; ++i ) {
18         if ( isalpha( texto[ i ] ) ) {
19             letra = tolower( texto[ i ] );
20             ++frequencia[ letra - 'a' ];
21         }
22     }
23
24     /* Exibe tabela */
25     printf( "\nLetra | Frequencia\n" );
26     printf( "-----|-----\n" );
27     for ( i = 0; i < 26; ++i ) {

```



```
28         if ( frequencia[ i ] > 0 ) {  
29             printf( "   %c   |   %d\n", 'a' + i,  
30                     frequencia[ i ] );  
31         }  
32  
33     return 0;  
34 }
```

16. Exercício 8.20 – Total de Palavras

Resposta detalhada

```
1  /* Ex8.20: conta_palavras.c */
2  #include <stdio.h>
3  #include <string.h>
4
5  int main( void )
6  {
7      char texto[ 500 ];
8      int  contador = 0;
9      char *token;
10
11     printf( "Digite o texto: " );
12     fgets( texto, 500, stdin );
13
14     token = strtok( texto, " \n\t" );
15     while ( token != NULL ) {
16         ++contador;
17         token = strtok( NULL, " \n\t" );
18     }
19
20     printf( "Total de palavras: %d\n", contador );
21     return 0;
22 }
```

17. Exercício 8.21 – Ordenação Alfabética de Strings

Resposta detalhada

```
1  /* Ex8.21: ordena_strings.c
2     Bubble sort de strings usando strcmp */
3  #include <stdio.h>
4  #include <string.h>
5  #define N 10
6
7  int main( void )
8  {
9      char *cidades[ N ] = {
10         "Sao Paulo", "Campinas", "Santos", "Sorocaba",
11         "Ribeirao Preto",
12         "Sao Jose do Rio Preto", "Bauru", "Marilia", "
13         Piracicaba", "Jundiai"
14     };
15
16     int i, j;
17     char *temp;
```

```
16
17     /* Bubble sort (Cap. 6) com strcmp */
18     for ( i = 0; i < N - 1; ++i ) {
19         for ( j = 0; j < N - 1 - i; ++j ) {
20             if ( strcmp( cidades[ j ], cidades[ j + 1
21                        ] ) > 0 ) {
22                 temp = cidades[ j ];
23                 cidades[ j ] = cidades[ j + 1 ];
24                 cidades[ j + 1 ] = temp;
25             }
26         }
27     }
28     printf( "Cidades em ordem alfabetica:\n" );
29     for ( i = 0; i < N; ++i ) {
30         printf( "    %s\n", cidades[ i ] );
31     }
32     return 0;
33 }
```

Detalhe importante: no array de ponteiros, trocamos os *ponteiros* (não os conteúdos das strings). Trocar ponteiros é $O(1)$; copiar strings seria $O(\text{comprimento})$.

18. Exercício 8.22 – ASCII (V/F)

Resposta detalhada

Consultando o Apêndice B (tabela ASCII), onde '0'=48, '9'=57, 'A'=65, 'Z'=90, 'a'=97, 'z'=122, '+'=43, '('=40, ')'=41:

- a) 'A' antes de 'B'? 65 < 66 – **Verdadeiro**.
- b) '9' antes de '0'? 57 > 48 – **Falso**. Os dígitos estão em ordem crescente, com '0' primeiro.
- c) Símbolos aritméticos antes dos dígitos? '+'=43, '-'=45, '*'=42, '/'=47, todos < 48 ('0'). **Verdadeiro**.
- d) Dígitos antes das letras? '9'=57, 'A'=65. **Verdadeiro**.
- e) Parêntese direito antes do esquerdo em ordem crescente? '('=40, ')'=41. Em ordem crescente, primeiro (, depois). Então) aparece *depois* do (, não antes. **Falso**.

19. Exercício 8.23 – Strings que começam com 'b'

Resposta detalhada

```
1  /* Ex8.23: começa_b.c */
2  #include <stdio.h>
3  #include <string.h>
4  #define N 10
5
6  int main( void )
7  {
8      char strings[ N ][ 50 ];
9      int i;
10
11     printf( "Digite %d strings:\n", N );
12     for ( i = 0; i < N; ++i ) {
13         scanf( "%s", strings[ i ] );
14     }
15
16     printf( "\nStrings começando com 'b':\n" );
17     for ( i = 0; i < N; ++i ) {
18         if ( strings[ i ][ 0 ] == 'b' || strings[ i ][
19             0 ] == 'B' ) {
20             printf( " %s\n", strings[ i ] );
21         }
22     }
23     return 0;
24 }
```

20. Exercício 8.24 – Strings terminando em 'ed'

Resposta detalhada

```
1  /* Ex8.24: termina_ed.c */
2  #include <stdio.h>
3  #include <string.h>
4  #define N 10
5
6  int main( void )
7  {
8      char strings[ N ][ 50 ];
9      int i, tam;
10
11     printf( "Digite %d strings:\n", N );
12     for ( i = 0; i < N; ++i ) scanf( "%s", strings[ i
13                                     ] );
14
15     printf( "\nStrings terminando em 'ed':\n" );
16     for ( i = 0; i < N; ++i ) {
17         tam = strlen( strings[ i ] );
18         if ( tam >= 2 &&
19             strings[ i ][ tam - 2 ] == 'e' &&
20             strings[ i ][ tam - 1 ] == 'd' ) {
21             printf( "  %s\n", strings[ i ] );
22         }
23     }
24     return 0;
25 }
```

21. Exercício 8.25 – Tabela ASCII Completa

Resposta detalhada

```
1  /* Ex8.25: ascii_table.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int codigo;
7      int c;
8
9      /* Versao 1: usuario digita um codigo */
10     printf( "Digite um codigo ASCII (0-255): " );
11     scanf( "%d", &codigo );
12     printf( "Codigo %d = '%c'\n", codigo, codigo );
13
14     /* Versao 2: tabela completa */
15     printf( "\nTabela completa de 32 a 126 (
        imprimiveis):\n" );
16     for ( c = 32; c <= 126; ++c ) {
17         printf( "%3d='%c' ", c, c );
18         if ( ( c - 31 ) % 8 == 0 ) printf( "\n" );
19     }
20
21     return 0;
22 }
```

O que acontece com códigos 0–31 e 128–255? Os de 0–31 são caracteres de controle (`\0`, `\t`, `\n`, beep, etc.) – alguns afetam o terminal de formas estranhas. Os 128–255 dependem da codificação ativa (ISO-8859, Windows-1252, UTF-8), produzindo caracteres acentuados ou inválidos.

22. Exercícios 8.26 a 8.33 – Reimplementar Funções da Biblioteca

Resposta detalhada

Visão geral: estes exercícios pedem para reimplementar funções da biblioteca padrão. São excelentes para entender *como* essas funções funcionam internamente.

Exemplo: 8.33 – strlen próprio:

```
1  /* Versao com subscritos */
2  size_t meu_strlen_v1( const char *s )
3  {
4      size_t i = 0;
5      while ( s[ i ] != '\0' ) ++i;
6      return i;
7  }
```

```
8
9  /* Versao com aritmetica de ponteiros */
10 size_t meu_strlen_v2( const char *s )
11 {
12     const char *p = s;
13     while ( *p != '\0' ) ++p;
14     return p - s; /* diferenca de ponteiros = numero
15                 de bytes */
16 }
```

Exemplo: 8.28 – strcpy próprio:

```
1  /* Com subscriptos */
2  char *meu_strcpy_v1( char *dest, const char *src )
3  {
4      int i = 0;
5      while ( src[ i ] != '\0' ) {
6          dest[ i ] = src[ i ];
7          ++i;
8      }
9      dest[ i ] = '\0';
10     return dest;
11 }
12
13 /* Com ponteiros (idiomatic C!) */
14 char *meu_strcpy_v2( char *dest, const char *src )
15 {
16     char *original = dest;
17     while ( ( *dest++ = *src++ ) != '\0' )
18         ; /* tudo na condicao */
19     return original;
20 }
```

Exemplo: 8.30 – strcmp próprio:

```
1  int meu_strcmp( const char *s1, const char *s2 )
2  {
3      while ( *s1 != '\0' && *s1 == *s2 ) {
4          ++s1; ++s2;
5      }
6      return *s1 - *s2; /* < 0, 0 ou > 0 */
7  }
```

Os exercícios 8.26 (funções <ctype.h>), 8.27 (conversões), 8.29 (E/S), 8.31 (busca) e 8.32 (memória) seguem a mesma filosofia: cada um aprofunda o domínio sobre uma família de funções específicas.

23. Exercício 8.34 – Análise de Texto

Resposta detalhada

```
1  /* Ex8.34: analise_texto.c
2     Tres analises diferentes em um unico programa */
3  #include <stdio.h>
4  #include <string.h>
5  #include <ctype.h>
6  #define MAX 1000
7
8  int main( void )
9  {
10     char texto[ MAX ];
11     int  freq_letra[ 26 ] = { 0 };
12     int  freq_tamanho[ 20 ] = { 0 };
13     int  i, tam;
14     char *token;
15     char copia[ MAX ];
16
17     printf( "Digite o texto (uma so linha): " );
18     fgets( texto, MAX, stdin );
19     strcpy( copia, texto );
20
21     /* (a) Frequencia de letras */
22     for ( i = 0; texto[ i ] != '\0'; ++i ) {
23         if ( isalpha( texto[ i ] ) ) {
24             ++freq_letra[ tolower( texto[ i ] ) - 'a'
25                             ];
26         }
27     }
28
29     printf( "\n== (a) Frequencia de letras ==\n" );
30     for ( i = 0; i < 26; ++i ) {
31         if ( freq_letra[ i ] > 0 ) {
32             printf( "%c: %d    ", 'a' + i, freq_letra[
33                 i ] );
34         }
35     }
36     printf( "\n" );
37
38     /* (b) Distribuicao de tamanhos */
39     token = strtok( texto, " .,:;\n" );
40     while ( token != NULL ) {
41         tam = strlen( token );
42         if ( tam < 20 ) ++freq_tamanho[ tam ];
43         token = strtok( NULL, " .,:;\n" );
44     }
45
46     printf( "\n== (b) Distribuicao de tamanhos ==\n" );
```



```
    );  
45     printf( "Tamanho | Ocorrencias\n" );  
46     for ( i = 1; i < 20; ++i ) {  
47         if ( freq_tamanho[ i ] > 0 ) {  
48             printf( "    %2d    | %d\n", i,  
                    freq_tamanho[ i ] );  
49         }  
50     }  
51  
52     /* (c) Lista de palavras unicas */  
53     printf( "\n=== (c) Palavras encontradas ===\n" );  
54     token = strtok( copia, " .,:;\n" );  
55     while ( token != NULL ) {  
56         printf( "    %s\n", token );  
57         token = strtok( NULL, " .,:;\n" );  
58     }  
59  
60     return 0;  
61 }
```

Nota: A análise mais sofisticada (item c com contagem de *palavras únicas*) requer uma estrutura de dados como tabela hash ou árvore (Cap. 12+). Aqui apresentamos uma versão simplificada listando todas as palavras encontradas.

24. Exercício 8.35 – Justificação de Texto

Resposta detalhada

Algoritmo:

- Ler palavra por palavra.
- Acumular palavras enquanto couberem em 65 colunas.
- Quando uma linha estiver cheia, distribuir os espaços extras uniformemente entre as palavras.
- Última linha: alinha à esquerda (sem distribuir espaços).

```

1  /* Esqueleto */
2  #define LARGURA 65
3
4  void justificar( char *palavras[], int n, int
   espacoExtra )
5  {
6      int gaps = n - 1;           /* numero de espacos
   entre palavras */
7      int base = espacoExtra / gaps;
8      int resto = espacoExtra % gaps;
9      int i, j;
10
11     for ( i = 0; i < n; ++i ) {
12         printf( "%s", palavras[ i ] );
13         if ( i < gaps ) {
14             int sp = base + 1;   /* pelo menos 1 espaco
   */
15             if ( i < resto ) ++sp; /* distribui
   sobras */
16             for ( j = 0; j < sp; ++j ) putchar( ' ' );
17         }
18     }
19     putchar( '\n' );
20 }

```

Curiosidade: A justificação de texto é um problema clássico em editoração eletrônica. Donald Knuth desenvolveu o algoritmo *Knuth-Plass* para o T_EX – usado para tipografar este próprio documento! – que produz justificação “ótima” globalmente, e não linha-por-linha.

25. Exercício 8.36 – Datas em Vários Formatos

Resposta detalhada

```

1  /* Ex8.36: formato_datas.c */
2  #include <stdio.h>
3
4  int main( void )

```

```
5 {
6     int dia, mes, ano;
7     const char *meses[] = {
8         "", "janeiro", "fevereiro", "marco", "abril",
9         "maio", "junho", "julho", "agosto",
10        "setembro", "outubro", "novembro", "dezembro"
11    };
12
13    printf( "Digite a data (DD/MM/AAAA): " );
14    scanf( "%d/%d/%d", &dia, &mes, &ano );
15
16    if ( mes >= 1 && mes <= 12 ) {
17        printf( "Formato extenso: %d de %s de %d\n",
18            dia, meses[ mes ], ano );
19    }
20    else {
21        printf( "Mes invalido!\n" );
22    }
23    return 0;
24 }
```

26. Exercício 8.37 – Proteção de Cheque

Resposta detalhada

```
1  /* Ex8.37: protecao_cheque.c */
2  #include <stdio.h>
3  #include <string.h>
4
5  int main( void )
6  {
7      double valor;
8      char    bufferValor[ 20 ];
9      char    protegido[ 20 ];
10     int      n, asteriscos, i;
11
12     printf( "Digite o valor do cheque: " );
13     scanf( "%lf", &valor );
14
15     /* Converte para string com 2 casas decimais */
16     sprintf( bufferValor, "%.2f", valor );
17     n = strlen( bufferValor );
18
19     /* Calcula quantos asteriscos para totalizar 9
20        chars */
21     asteriscos = 9 - n;
22     if ( asteriscos < 0 ) asteriscos = 0;
23
24     /* Constroi string protegida: asteriscos + valor
25        */
26     for ( i = 0; i < asteriscos; ++i ) protegido[ i ]
27         = '*';
28     protegido[ asteriscos ] = '\0';
29     strcat( protegido, bufferValor );
30
31     printf( "Cheque: %s\n", protegido );
32     printf( "          123456789\n" );
33
34     return 0;
35 }
```

Exemplo: 99,87 → ****99.87; 11230,60 → 11230.60 (já completo).

27. Exercício 8.38 – Valor de Cheque por Extenso

Resposta detalhada

Esquema: converter *centenas + dezenas + unidades + centavos* em palavras. Tabelas auxiliares para cada caso.

```

1  /* Ex8.38: cheque_extenso.c -- esqueleto */
2  const char *unidades[] = {
3      "", "um", "dois", "tres", "quatro", "cinco",
4      "seis", "sete", "oito", "nove"
5  };
6  const char *adolescentes[] = {
7      "dez", "onze", "doze", "treze", "catorze",
8      "quinze", "dezesesseis", "dezessete", "dezoito", "
9      dezenove"
10 };
11 const char *dezenas[] = {
12     "", "", "vinte", "trinta", "quarenta", "cinquenta"
13     ,
14     "sessenta", "setenta", "oitenta", "noventa"
15 };
16 const char *centenas[] = {
17     "", "cento", "duzentos", "trezentos", "
18     quatrocentos",
19     "quinhentos", "seiscentos", "setecentos", "
20     oitocentos", "novecentos"
21 };
22
23 /* Funcao recursiva para converter 0-999 em palavras
24 */
25 void converte( int n, char *resultado ) {
26     if ( n == 100 ) {
27         strcat( resultado, "cem" );
28         return;
29     }
30     if ( n >= 100 ) {
31         strcat( resultado, centenas[ n / 100 ] );
32         strcat( resultado, " e " );
33         n %= 100;
34     }
35     /* ... continuar para dezenas e unidades ... */
36 }

```

Atenção: as regras de português têm peculiaridades (“cem” vs. “cento e cinco”, “e” entre dezenas e unidades, etc.) que tornam essa conversão mais complexa do que parece. Esse exercício é excelente para praticar muitos `if/else` em strings.

28. Exercício 8.39 – Código Morse

Resposta detalhada

```

1  /* Ex8.39: morse.c */
2  #include <stdio.h>
3  #include <ctype.h>
4  #include <string.h>
5
6  const char *morse[] = {
7      ".-", "-...", "-.-.", "-..", ".",
8      "...", "-.-", "...", "..", "-.-.-",
9      "-.", "...", "--", "--", "--",
10     "...", "-.-", "-.", "...", "-",
11     "-.", "...", "-.-", "-.-", "-.-.-", "-.-."
12 };
13
14 const char *morseDig[] = {
15     "-.-.-", "-.-.-", "-.-.-", "-.-.-", "-.-.-",
16     "-.-.-", "-.-.-", "-.-.-", "-.-.-", "-.-.-"
17 };
18
19 void codifica( const char *frase )
20 {
21     int i;
22     for ( i = 0; frase[ i ] != '\0'; ++i ) {
23         char c = toupper( frase[ i ] );
24         if ( c >= 'A' && c <= 'Z' ) {
25             printf( "%s ", morse[ c - 'A' ] );
26         }
27         else if ( c >= '0' && c <= '9' ) {
28             printf( "%s ", morseDig[ c - '0' ] );
29         }
30         else if ( c == ' ' ) {
31             printf( "   " ); /* 3 espaços entre
32                               palavras (mais o anterior) */
33         }
34     }
35     printf( "\n" );
36
37 int main( void )
38 {
39     char frase[ 200 ];
40
41     printf( "Digite uma frase: " );
42     fgets( frase, 200, stdin );
43     frase[ strcspn( frase, "\n" ) ] = '\0';
44
45     printf( "Morse: " );

```

```
46     codifica( frase );
47
48     return 0;
49 }
```

A decodificação (Morse $\rightarrow$ texto) é o processo inverso: separar por espaços duplos (palavras), depois por espaços simples (letras), e mapear de volta com busca em uma tabela.

29. Exercício 8.40 – Conversor Métrico

Resposta detalhada

Estratégia: cada unidade tem (a) uma *categoria* (comprimento, massa, volume, etc.) e (b) um *fator* em relação a uma unidade-base. Conversões só fazem sentido entre unidades da mesma categoria.

```

1  /* Esqueleto */
2  typedef struct {
3      const char *nome;
4      int categoria; /* 0=comprimento, 1=massa, 2=
5                      volume */
6      double fator; /* em relacao a unidade-base da
7                      categoria */
8  } Unidade;
9
10 Unidade tabela[] = {
11     /* Comprimento, base = metro */
12     { "metro", 0, 1.0 },
13     { "centimetro", 0, 0.01 },
14     { "polegada", 0, 0.0254 },
15     { "pe", 0, 0.3048 },
16     /* Massa, base = kg */
17     { "quilograma", 1, 1.0 },
18     { "grama", 1, 0.001 },
19     { "libra", 1, 0.4535 },
20     /* Volume, base = litro */
21     { "litro", 2, 1.0 },
22     { "quarto", 2, 0.9463 },
23     /* ... */
24 };
25
26 double converte( double valor, const char *origem,
27                  const char *destino ) {
28     /* buscar origem e destino na tabela; checar mesma
29        categoria;
30        converter valor para base e da base para
31        destino */
32 }

```

30. Exercício 8.41 – Cartas de Cobrança

Resposta detalhada

Estratégia: 5 templates de carta (do mais gentil ao mais ameaçador), com “placeholders” substituídos por dados do devedor.

```

1  const char *templates[ 5 ] = {

```



```
2      /* 1 mes: amigavel */
3      "Prezado(a) %s,\nNotamos que sua conta %d, no
        valor de R$ %.2f\n"
4      "esta com 1 mes em aberto. Por favor, considere
        regularizar.\n",
5
6      /* 2 meses: mais firme */
7      "Prezado(a) %s,\nSua conta %d, no valor de R$ %.2f
        , esta vencida\n"
8      "ha 2 meses. Solicitamos pagamento imediato.\n",
9
10     /* ... ate cinco niveis ... */
11 };
12
13 int main( void ) {
14     char nome[ 100 ], endereco[ 200 ];
15     int  conta, atraso;
16     double valor;
17     /* ... ler dados, escolher template apropriado,
        imprimir ... */
18 }
```

31. Exercício 8.42 – Gerador de Palavras Cruzadas

Resposta detalhada

Por que é um projeto difícil? Envolve:

- **Representação:** grade $N \times N$ como matriz de char, com '.' para vazio e '#' para bloqueio.
- **Dicionário:** milhares de palavras em arquivo (Cap. 11), indexadas por comprimento e padrão (ex.: "C_S_").
- **Backtracking:** tenta encaixar palavras horizontal/verticalmente; retrocede se nenhuma cabe.
- **Geração de dicas:** requer banco de dados de definições para cada palavra.

Este é um **projeto de final de período** – pode levar 2–3 semanas em laboratório. Recomenda-se começar com versões simplificadas: grade pequena (5×5), dicionário compacto (100 palavras), sem dicas.